POO C#

•••

GT3 - Kévin Roussel

Qu’est-ce qu’un champs (field) ?

Quelle est la différence entre un tableau (array) et une liste (list) ?

Quelle est la différence entre
une classe (class) et
une structure (struct) ?

Quel est le plus gros problème du type float ?

indice : epsilon

Qu’est-ce qu’une interface ?

Objectif de la semaine :

Créer un nouveau type et l'exploiter

Health :

Construct

Field

Property

Method

Event

\+ héritage, exceptions, optimisation

Préparation

Conseils :

Utilisez Visual Studio 2022

Versionnez votre code

Le projet de la semaine :

Jeu en console

...

Utiliser la classe Console
Rechercher le potentiel de cette classe
Formaliser son code, le garder consistant et maintenable dans le temps

Quelques exemples :

Qu’est-ce qu’un objet ?

Un ensemble de champs et de méthodes

Le but : rassembler un ensemble cohérent de fonctionnalités.

- Organisation sous forme d'objet, on fait en sorte de ne lui dédier qu'une responsabilité.
- Sur Unity (ou autre moteur) on travaille majoritairement en composition et moins en héritage
- Chaque classe a sa responsabilité et c'est l'ensemble des composants qui va donner une cohérence à une entité (joueur, PNJ, interactable etc)

Une fois l'ensemble des composants mis en place :

- On doit gérer la communication entre chaque composants, on parle alors d'architecture, les composants ont des relations entre eux.

- Ils peuvent se donner de l'information (property)
- Ils peuvent s'avertir d'un évènement (event)
- Ils peuvent utiliser une fonctionnalité d'un autre composant (method)
- Un composant conserve également des informations précieusement (field)

Parlons un peu de console

Les méthodes essentielles :

Console.Write

Console.Read (et ReadKey)

Console.SetPosition

Console.BackgroundColor / ForegroundColor

Formalisation de notre démarrage : Main

```
class Program
{
    static void Main()
    {
        Console.WriteLine("Hello World !");
        Console.Read();
    }
}
```

Quelques bases

Classe

Concept

Rassemble un ensemble d’information et de fonctionnalité

Une classe est un plan de construction, on écrit le contenu que va contenir chaque instance. **Une instance** est unique et peut évoluer indépendamment.

Type = Classe = Plan de construction

Instance = this = Composant évoluant au runtime

Concept

Les instances possèdent donc les mêmes champs et méthodes mais leur contenu change de façon indépendante (pour conserver une information au niveau du type, on utilisera static)

Visibilité

Dans une classe, les champs et méthodes ont 2 visibilités principales (+2 bonus).

Permet d'ouvrir nos fonctionnalités aux autres composants (ou à l'inverse, permet de cacher nos mécanismes qui ne concernent que nous)

L'idée n'est de proposer aux autres classes que le stricte minimum. On propose une surface d'appel qui respecte les besoins des autres composants, le reste c'est notre mécanique interne.

Visibilité (bonus)

public : dispo à tous

private : dispo uniquement pour notre classe

protected : héritage

internal : assembly

Champs

[Type] label;

int : nombre entier

float : nombre à virgule flottante (ajouter le ‘f’)

string : chaine de caractère

bool : vrai/faux (true/false)

Vector3 : position ou direction (Unity)

Color : couleur (Unity)

ex : int _maVie; float _monArgent; string _monNom; Vector3 _maDirection;

Version complète : [Attributs] [Visibilité] [Type] label;

L’information

Nos champs nous permettent de conserver des informations qui doivent rester dans le temps.

Convention de nommage : commencer par un ‘_’ puis camelCase

Ex : _currentLife, _maxLife

/!\ Attention : une variable n’est pas un champs (dispo qu’au sein d’une fonction)

Pas de champs public !

- On dévoile toutes nos informations vitales
- On autorise d'autres composants à modifier nos valeurs essentielles
  - Une vie à -12 ? Merci notre cher.e stagiaire
  - Une vie supérieure à notre valeur de vie max ? Oups
- Nos champs, c'est notre jardin secret
  - Il n'y a QUE notre composant qui doit y avoir accès.
  - ... Et si on veut proposer aux gens d'accéder à ces valeurs, on mettra en place des propriétés
  - On peut choisir si on veut proposer ou pas la configuration des valeurs initiales via l'editeur. [SerializeField]

Sur Unity

[SerializeField] permet au game designer de renseigner une valeur de départ pour le composant ... Mais il peut faire des erreurs ou il peut avoir la flemme de renseigner des valeurs initiales

On peut mettre une information sur un champs, une méthode ou une classe via l'utilisation d'un attribut. Il existe tout un lot d'attribut déjà existant et on peut également créer les nôtres au besoin (un peu plus avancé).

Notre métier

- Créer des composants qui vont fitter avec les besoins énoncé par les GD
- Organiser les entités pour qu'elles puissent interagir facilement
- Ouvrir des outils et champs aux GD pour qu'ils puissent configurer les entités (vous avez pas envie de faire ça)
- Créer du tooling pour rendre le travail de vos collègues plus agréable.

Les petits mot clés sympas :

var

?.

??

new()

dynamic (juste pour la culture à ne jamais utiliser)

Health

Mettre en place nos champs

MaxLife - dispo pour le GD - forcément positif - valeur initiale : 100

CurrentLife - interne au composant - évolue dans le temps - rendre le champ visible au runtime

Méthodes

Concept

Rassemble une série d’instructions formant une fonctionnalité.

Variables, assignations, conditions, boucles, appels de fonction

Une méthode peut être enclenché par un autre composant (fonction publique)

Une méthode peut également être dispo que pour notre fonctionnement interne (fonction privé)

2 catégories de méthodes

- Fonction :
  - Ensemble d'instruction aboutissant à un résultat donné.
  - ex : Mathf.Abs, Mathf.Min etc
- Procédure :
  - Ensemble d'instruction ne retournant pas de résultat
  - ex : Debug.Log

Variable

[TYPE] labelDeLaVariable = {Valeur initiale};

- Une information temporaire au sein d'une fonction/méthode
- camelCase simple

Faisons la distinction entre champs et variable

- un champs commence par un '_' (underscore) et est écrit en camelCase
  - _monChamps
- une variable est écrit en camelCase
  - maVariable

Quand on modifie une information labélisé avec un underscore on sait qu'on touche à un champs

Scope

Une variable n'est disponible qu'au sein d'une fonction. À la fin de l'execution de la fonction la variable n'existe plus.

Elle n'existe que dans la porté de la fonction (scope)

Un scope = entre 2 accolades

Dès qu'il y a des accolades, il y a un scope (classe, méthodes, conditions, boucles etc)

Un champs étant déclaré au sein d'une classe, le champs est persistant et est dispo dans les méthodes

Signature de fonction

Fonctions :

[Type de retour] LabelDeLaFonction ()

Fonction retournant un résultat

[Type de retour] LabelDeLaFonction (**type** labelParam1, **type** labelParam2 etc)

Autant de paramètres que l'on veut. ça va nous créer une variable automatiquement.

Procédures :

void LabelDeLaFonction()

void LabelDeLaFonction(type labelParam1 etc)

Pas de retour, on peut demander des paramètres (ex: Debug.Log)

Signature : un mode d'emploi

- Permet de savoir ce que l'on va recevoir à la suite d'un appel
- Consultez la doc pour avoir plus d'informations sur les fonctions que vous utilisez

Health

On veut rendre dispo une méthode pour perdre des points de vie

Une méthode pour gagner des points de vie également

On a une méthode pour mourir qui n'est appellé qu'en interne (le composant Health décide de quand appeller "Die")

Implémenter Health et faire un autre composant qui va exploiter les méthodes de Health. Utiliser NaughtyAttributes pour rendre le tout facilement accessible pour les tests

C’est le moment de parler des Test Unitaires

- Le but : prouver que votre code fonctionne
- Votre production doit être robuste et gérer même les cas “exotiques” qui viendrait d’une mauvaise utilisation de vos classes.
    - Ex : maVie.TakeDamage(- 12); // Ici on prend des dégâts négatifs, comment on doit gérer ce cas ?

Les tests unitaires sont un projet à côté de votre logiciel

Il n'a aucun intérêt de se retrouver dans le build final, ce n'est qu'un outil pour les développeurs pour prouver que votre taff fonctionne.

Dans un contexte avec une hiérarchie de travail plus élaboré, votre supérieur devrait valider le code que vous poussez en production. Comment peut-il savoir que vous n'avez pas fait d'erreur ? Les tests unitaires sont une preuve *en plus* que l'on peut avoir confiance dans votre production.

Création d’un projet de test unitaire en live et premier test.

Voici votre premier projet avec des test unitaires à compléter

La première fois avec des TU c'est toujours difficile, c'est tout à fait normal de ne pas comprendre l'ensemble de la démarche, ni de comprendre comment en créer.

Dans un premier temps je vous donnerais des TU, donc les tests seront déjà fait et votre code devra valider les tests.

Une fois plus à l'aise on va pouvoir prendre l'initiative de créer des TU pour prouver que ce que l'on a fait est robuste.